

Multi-Agent RAG — Operations Runbook

Everything needed to **run the multi-agent RAG locally**, **deploy it to a server**, and **scale it with production-grade strategies**. Every step lists the exact command. Pairs with the architecture guide ([multi-agent-rag-guide.html](#)); this document is the hands-on operator's manual.

Document type: Run · Deploy · Scale

Mode: `AGENT_MODE=multiagent`

FastAPI · gunicorn · Docker · K8s

Verified live (Azure OpenAI)

Contents

- | | |
|--|--|
| 1. What you are running | 8. Kubernetes |
| 2. Prerequisites | 9. Scaling strategies |
| 3. Run locally (step by step) | 10. Observability & health |
| 4. Using it — CLI & API | 11. Security hardening |
| 5. Configuration reference | 12. Troubleshooting |
| 6. Containers — Docker & Compose | 13. Production readiness checklist |
| 7. Production deployment | |

1. What you are running

The multi-agent RAG mode is a LangGraph supervisor pipeline — **supervisor** → **retrieval** → **synthesis** → **critic** — selected by a single flag. It shares the project's entire production scaffolding (FastAPI service, API-key auth, rate limiting, request timeout, durable checkpointers, Prometheus metrics, Docker/K8s artifacts) with the default `single` agent. Switching modes is a config change, never a code change.

Layer	Component	Where
Orchestration	Supervisor RAG graph (4 nodes, bounded revision loop)	<code>multiagent/graph.py</code>
Retrieval	Hybrid dense+BM25 with RRF fusion, optional cross-encoder rerank, multi-query expansion	<code>rag/</code>
Service facade	<code>MultiAgentRAG</code> (<code>ask</code> / <code>stream</code> / <code>health</code> / <code>close</code>)	<code>multiagent/service.py</code>
Selector	<code>build_agent_service()</code> picks single vs multiagent	<code>service_factory.py</code>
Serving	FastAPI + gunicorn/uvicorn workers	<code>api/app.py</code>
State	Checkpointers: memory · sqlite · postgres	<code>agents/checkpointer.py</code>

One mental model: the multi-agent graph replaces the agent's "brain," but the *request path* (auth → rate-limit → timeout → service → metrics) is identical to single mode. Anything you know about operating the API applies unchanged.

2. Prerequisites

Requirement	Version / note	Check
Python	3.13	<code>python --version</code>
uv (package manager)	latest	<code>uv --version</code>
LLM provider key	one of: Groq · OpenAI · Azure · Anthropic · Gemini · Ollama (local)	set in <code>.env</code>
Disk	~300 MB free (embedding model + optional reranker download on first use)	<code>df -h .</code>
Docker (optional)	for container/compose runs	<code>docker --version</code>
Postgres (optional)	for durable multi-replica state — provided by Compose	—

Install uv if needed:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

3. Run locally (step by step)

1 Clone & install dependencies

```
cd "Study/Agentic AI/langchain"
uv sync                # core deps (multi-agent RAG included)
uv sync --extra postgres # add only if you will use CHECKPOINTINTER=postgres
```

All RAG dependencies (`rank-bm25`, embeddings, LangGraph) are in the core install. The cross-encoder reranker model is downloaded lazily only when `RERANK_ENABLED=true`.

2 Configure the environment

```
cp .env.example .env
```

Edit `.env` and set, at minimum, the mode and a provider key:

```
# --- pick the multi-agent RAG pipeline ---
AGENT_MODE=multiagent

# --- pick ONE provider (example: Groq, free key) ---
LLM_PROVIDER=groq
GROQ_API_KEY=gsk... # leave LLM_MODEL unset → provider default applies

# --- advanced RAG knobs (sensible defaults shown) ---
RETRIEVER_TOP_K=4
RETRIEVER_FETCH_K=10
MULTIQUERY_ENABLED=true
MULTIQUERY_COUNT=3
RERANK_ENABLED=false
MAX_REVISIONS=1
```

.env hygiene: never commit `.env` (it is gitignored). No trailing commas or quotes on values — a trailing comma on an Azure endpoint/key is a known cause of `404 Resource not found`. Keys are held as `SecretStr` and never logged.

3 Build the knowledge base (ingest)

The multi-agent retriever reads from the same Chroma index as single mode. Drop your documents (`.md` , `.txt` , `.pdf`) into `data/knowledge_base/` , then:

```
uv run research-agent ingest          # first run downloads the embedding model (~80 MB)
uv run research-agent ingest --force  # rebuild after changing documents
```

Hybrid retrieval needs the corpus: BM25 is built in-memory from the indexed chunks at startup, and dense search queries the Chroma store. If you skip ingest, the supervisor can still answer via the *direct* (no-retrieval) path, but you get no grounded citations.

3 Verify the configuration loaded correctly

```
AGENT_MODE=multiagent uv run research-agent ask "ping" 2>&1 | head
# startup log should read: Initializing MultiAgentRAG (provider=..., model=..., rerank=..., multiquery=...)
```

4 Ask a grounded question

```
AGENT_MODE=multiagent uv run research-agent ask "What is RAG? Cite your sources."
```

Expected internal flow (visible in logs / stream): `[supervisor] route=retrieve → [retrieval] N chunks → [synthesis] → [critic] grounded=True` , then a cited answer.

5 Or chat interactively (with memory)

```
AGENT_MODE=multiagent uv run research-agent chat -t my-session
```

The `-t/--thread` id keeps separate conversation memories; with a durable checkpoint it survives restarts.

Tip — set the mode once. Rather than prefixing every command, put `AGENT_MODE=multiagent` in `.env` . The CLI, chat, and API all read it.

4. Using it — CLI & API

CLI

Command	Description
<code>research-agent ask "<q>"</code>	Answer one question and exit.
<code>research-agent chat</code>	Interactive REPL with thread memory.
<code>research-agent ingest [-f]</code>	Build / rebuild the knowledge base.

HTTP API

Run the same multi-agent pipeline as a service. The agent is built **once at startup** (lifespan) and shared across requests:

```
# single process (dev)
AGENT_MODE=multiagent uv run research-agent-api      # → http://localhost:8000
(docs at /docs)

# multi-worker (production)
AGENT_MODE=multiagent gunicorn agentic_research_agent.api.app:app \
  -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000 --timeout 120
```

Endpoint	Method	Purpose
<code>/v1/ask</code>	POST	Answer a question → <code>AgentResponse</code> (auth + rate-limited).
<code>/v1/stream</code>	POST	Stream the run as Server-Sent Events (per-node step labels, then the answer).
<code>/health/live</code>	GET	Liveness probe.
<code>/health/ready</code>	GET	Readiness — KB reachable + LLM configured.
<code>/metrics</code>	GET	Prometheus metrics.

```
# ask
curl -X POST http://localhost:8000/v1/ask \
  -H "Content-Type: application/json" -H "X-API-Key: $KEY" \
  -d '{"question": "What is hybrid retrieval? Cite sources.", "thread_id": "demo"}'

# stream (see each agent fire in real time)
curl -N -X POST http://localhost:8000/v1/stream \
  -H "Content-Type: application/json" -H "X-API-Key: $KEY" \
  -d '{"question": "Explain RRF.", "thread_id": "demo"}'
```

As a library

```
from agentic_research_agent import build_agent_service
from agentic_research_agent.config.settings import get_settings

svc = build_agent_service(get_settings()) # honours AGENT_MODE
print(svc.ask("Explain reranking. Cite sources.").answer)
```

5. Configuration reference

Everything is environment-driven (pydantic-settings). The knobs most relevant to multi-agent RAG:

Variable	Default	Effect
AGENT_MODE	single	Set <code>multiagent</code> to run this pipeline.
LLM_PROVIDER	groq	groq · openai · azure · anthropic · gemini · ollama.
LLM_MODEL	<i>unset</i>	Leave unset → per-provider default resolves automatically.
RETRIEVER_TOP_K	4	Final chunks kept after fusion/rerank.
RETRIEVER_FETCH_K	10	Candidates pulled per retriever before fusion. ↑ recall, ↑ cost.
MULTIQUERY_ENABLED	true	Expand the question into several search queries.
MULTIQUERY_COUNT	3	Number of expansions (more = more LLM calls + recall).
RERANK_ENABLED	false	Cross-encoder rerank. ↑ precision, downloads a model, adds CPU latency.
RERANK_MODEL	ms-marco-MiniLM-L-6-v2	Cross-encoder used when reranking is on.
MAX_REVISIONS	1	Critic-triggered re-retrieval rounds (bounds latency/cost).
CHECKPINTER	memory	memory · sqlite · postgres (durability of conversation threads).
LLM_MAX_RETRIES	2	Provider retries on 429/5xx.

Variable	Default	Effect
<code>API_KEYS</code>	<i>empty</i>	Comma-separated; empty disables auth (dev only).
<code>RATE_LIMIT_PER_MINUTE</code>	<code>60</code>	Per key / client IP.
<code>REQUEST_TIMEOUT_SECONDS</code>	<code>120</code>	Per-request server timeout.

Tuning intuition. Recall problems? raise `RETRIEVER_FETCH_K` and keep `MULTIQUERY_ENABLED=true`. Precision problems (right docs buried)? turn on `RERANK_ENABLED`. Hallucination/ungrounded answers? raise `MAX_REVISIONS` to `2`. Cost/latency too high? lower `MULTIQUERY_COUNT`, set `MAX_REVISIONS=0`, and use a cheaper supervisor model.

6. Containers — Docker & Compose

Single image

The multi-stage `Dockerfile` installs with uv, runs as non-root (uid 10001), pre-warms the embedding model, and serves via gunicorn with 4 uvicorn workers and a `/health/ready` healthcheck.

```
docker build -t agentic-research-agent:0.1.0 .

docker run --rm -p 8000:8000 \
  -e AGENT_MODE=multiagent \
  -e LLM_PROVIDER=groq -e GROQ_API_KEY=$GROQ_API_KEY \
  -e API_KEYS=$API_KEYS \
  agentic-research-agent:0.1.0
```

Compose — API + Postgres (durable state)

The bundled `docker-compose.yml` wires the API to a `pgvector/pgvector:pg16` Postgres for the durable checkpointer. Add the mode flag:

```
# in .env (read by compose): set GROQ_API_KEY, API_KEYS, and:
AGENT_MODE=multiagent

docker compose up --build          # API on :8000, Postgres with healthcheck + named
volume
```

The compose `api` service already sets `CHECKPOINTER=postgres` and a `POSTGRES_DSN` pointing at the bundled database, and waits for it to pass `pg_isready` before starting.

Mode in compose. The provided file passes through `${LLM_PROVIDER}`, `${GROQ_API_KEY}`, `${API_KEYS}`. Add `AGENT_MODE: ${AGENT_MODE:-single}` under the `api` service `environment:` block (or export it in your shell) so the container starts in multi-agent mode.

7. Production deployment

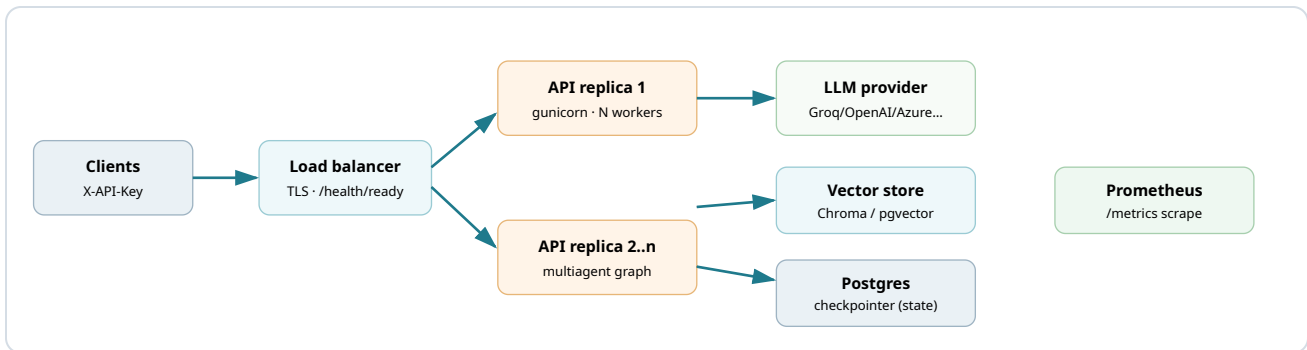


Figure 1 — Stateless API replicas behind a load balancer. Conversation state is externalized to Postgres so any replica can serve any thread. The LLM provider and vector store are shared backing services.

7.1 The golden rule: externalize state

The API process is stateless *only* if conversation memory lives outside it. With the default `CHECKPOINTER=memory`, threads are lost on restart and not shared across replicas. For any multi-replica deployment:

```
CHECKPOINTER=postgres
POSTGRES_DSN=postgresql://user:pass@db-host:5432/agent
# build the image / install with the postgres extra: uv sync --extra postgres
```

For a single durable node, `CHECKPOINTER=sqlite` with `SQLITE_PATH` on a persistent volume is enough.

7.2 Bare-metal / VM with gunicorn + systemd

```
# /etc/systemd/system/research-agent.service
[Unit]
Description=Agentic Research Agent (multi-agent RAG)
After=network.target

[Service]
User=appuser
WorkingDirectory=/opt/agentic-research-agent
EnvironmentFile=/opt/agentic-research-agent/.env # holds
AGENT_MODE=multiagent, keys, DSN
```

```
ExecStart=/opt/agentic-research-agent/.venv/bin/gunicorn \
  agentic_research_agent.api.app:app \
  -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000 --timeout 120
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now research-agent
sudo systemctl status research-agent
curl -fsS http://localhost:8000/health/ready
```

Put Nginx/Caddy in front for TLS termination and a sane `proxy_read_timeout` (`> REQUEST_TIMEOUT_SECONDS`; SSE streaming needs buffering off).

7.3 Worker sizing

The multi-agent graph is **I/O-bound** — most wall-clock is spent waiting on LLM and retrieval calls, not CPU (unless reranking is on). Start with `-w = 2 × cores` and adjust by watching latency and provider rate limits. With `RERANK_ENABLED=true` the cross-encoder is CPU-bound; size workers to physical cores and keep the model resident (built once at startup).

8. Kubernetes

A minimal, production-shaped Deployment + Service + HPA. State is in Postgres (a managed instance or in-cluster), secrets in a `Secret`, autoscaling on CPU.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: research-agent
spec:
  replicas: 3
  selector: { matchLabels: { app: research-agent } }
  template:
    metadata: { labels: { app: research-agent } }
    spec:
      containers:
        - name: api
          image: registry.example.com/agentic-research-agent:0.1.0
          ports: [{ containerPort: 8000 }]
          env:
            - { name: AGENT_MODE, value: "multiagent" }
            - { name: LLM_PROVIDER, value: "groq" }
            - { name: CHECKPOINTER, value: "postgres" }
            - name: GROQ_API_KEY
              valueFrom: { secretKeyRef: { name: agent-secrets, key: groq-api-key } }
            - name: API_KEYS
              valueFrom: { secretKeyRef: { name: agent-secrets, key: api-keys } }
            - name: POSTGRES_DSN
              valueFrom: { secretKeyRef: { name: agent-secrets, key: postgres-dsn } }
      resources:
        requests: { cpu: "500m", memory: "512Mi" }
        limits: { cpu: "2", memory: "2Gi" }      # raise memory if
RERANK_ENABLED=true
      readinessProbe:
        httpGet: { path: /health/ready, port: 8000 }
        initialDelaySeconds: 20
        periodSeconds: 10
      livenessProbe:
        httpGet: { path: /health/live, port: 8000 }
```

```

        initialDelaySeconds: 15
        periodSeconds: 20
    ---
apiVersion: v1
kind: Service
metadata: { name: research-agent }
spec:
  selector: { app: research-agent }
  ports: [{ port: 80, targetPort: 8000 }]
  ---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: research-agent }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: research-agent }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
}

```

Probe semantics matter. `/health/ready` checks the knowledge base is reachable and the LLM is configured — a replica that can't retrieve is pulled from rotation rather than serving broken answers. `/health/live` only restarts a wedged process. Give `initialDelaySeconds` enough room for the embedding model to load at startup.

```

kubectl apply -f k8s.yaml
kubectl rollout status deploy/research-agent
kubectl get hpa research-agent -w

```

9. Scaling strategies

Scale along the axis that is actually the bottleneck. For this pipeline they are, in order of likelihood: **LLM throughput** → **retrieval store** → **reranker CPU** → **state store**.

9.1 Horizontal (the default)

Do	Why
Run replicas behind a load balancer; autoscale on CPU/latency (HPA above).	The API is stateless once state is in Postgres — replicas are interchangeable.
Externalize state to Postgres before adding replicas.	Otherwise thread memory is per-pod and lost on rescheduling.
Set sticky-session <i>off</i> .	Any replica can serve any <code>thread_id</code> because state is shared.

9.2 LLM is usually the real limit

Each multi-agent run makes several model calls (supervisor + query expansion × `MULTIQUERY_COUNT` + synthesis + critic, × revisions). Provider rate limits (TPM/RPM) cap you long before CPU does. Strategies:

- **Right-size the calls.** Lower `MULTIQUERY_COUNT` and `MAX_REVISIONS`; use a *cheap, fast model for the supervisor/critic* and a stronger one for synthesis (cheap routing, expensive writing).
- **Spread load.** Higher provider tier, multiple keys/deployments, or a gateway that load-balances across them. `LLM_MAX_RETRIES` already absorbs transient 429s.
- **Cache.** Add a semantic/exact response cache (e.g. Redis) keyed on normalized question + retrieved-doc set; short-circuit identical or near-identical questions. (Roadmap item — a clean seam exists at the service facade.)

9.3 Retrieval store

Stage	Store	Trade-off
Dev / single	Chroma (local, default)	Zero infra; index lives on the pod's disk — not shared.

Stage	Store	Trade-off
node		
Multi-replica	pgvector (shared Postgres) or Qdrant	One shared index; replicas don't each rebuild. Swap behind the <code>dense_search</code> seam in <code>build_hybrid_retriever</code> .
Large corpus	Managed vector DB (Qdrant/Weaviate/pgvector)	Horizontal index scaling, filtering, persistence; network hop per query.

BM25 note at scale. The sparse half currently builds an in-memory BM25 index from the corpus at startup. That is fine to tens of thousands of chunks. For larger corpora, move sparse retrieval to the store (e.g. Postgres full-text or an OpenSearch hybrid index) behind the same `HybridRetriever` interface so RRF fusion is unchanged.

9.4 Reranker placement

The cross-encoder is CPU-heavy and loaded per process. If you enable it broadly: pin workers to physical cores, raise pod memory limits, and consider running the reranker as a *separate service* (or on GPU nodes) that the retriever calls — the `Reranker` protocol makes this a drop-in swap.

9.5 State store

Postgres handles checkpointer writes comfortably; it is rarely the bottleneck. If it becomes one: connection pooling (PgBouncer), a read replica is not needed (writes dominate), and prune old threads on a schedule.

9.6 Vertical

More CPU only helps if reranking is on or you serve many concurrent CPU-bound requests; more memory helps with large embedding/reranker models and bigger in-memory BM25 indexes. For a pure LLM-bound workload, scale out (replicas), not up.

Symptom	First lever
p95 latency high, CPU low	LLM-bound → cheaper supervisor model, fewer multiqueries/revisions, cache.

Symptom	First lever
429s from provider	Higher tier / multiple keys / gateway; back-pressure via rate limit.
CPU pegged	Reranking on → scale out or offload reranker; raise replicas via HPA.
Answers lose memory across requests	Switch off <code>memory</code> checkpointer → Postgres.
Recall poor / wrong docs	↑ <code>RETRIEVER_FETCH_K</code> , enable rerank, shared/better vector store.

10. Observability & health

- **Metrics** — scrape `/metrics` (Prometheus). Track request rate, latency histograms, error rate, in-flight requests.
- **Health** — `/health/ready` for the load balancer/probe; `/health/live` for restart-only liveness.
- **Structured logs** — JSON in production. Each run logs `multiagent_run_started/finished` with `run_id`, `thread_id`, `duration_ms`, `route`, and `docs` count — enough to trace any answer.
- **Tracing** — set `LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY` to capture the full agent graph (every node, prompt, and LLM call) in LangSmith. Invaluable for debugging routing and grounding.

```
# quick local Prometheus scrape config
scrape_configs:
  - job_name: research-agent
    metrics_path: /metrics
    static_configs:
      - targets: ["research-agent:80"]
```

Key SLO signals for this pipeline: synthesis latency (dominant), critic revision rate (high = retrieval quality issue), and provider 429 rate (capacity).

11. Security hardening

- **Auth on** — set `API_KEYS` in every non-dev environment; empty means open. Rotate keys via the secret store.
- **Secrets at runtime** — inject via env/Secret, never bake into the image or commit `.env`. Keys are held as `SecretStr` and never logged.
- **Rate limit** — keep `RATE_LIMIT_ENABLED=true`; tune per-key. Protects you and your provider quota.
- **Timeouts** — `REQUEST_TIMEOUT_SECONDS` caps a wedged run; align upstream proxy timeouts.
- **CORS** — set `CORS_ORIGINS` to an explicit allow-list in production (not `*`).
- **Non-root container** — the image already runs as uid 10001; keep it that way and run read-only root FS where possible.
- **TLS** — terminate at the load balancer / ingress; never expose plain HTTP publicly.

12. Troubleshooting

Symptom	Likely cause	Fix
Answers have no citations / always "direct"	Knowledge base empty or not ingested	<code>uv run research-agent ingest --force</code> ; confirm files in <code>data/knowledge_base/</code> .
<code>404</code> <code>model_not_found</code>	Wrong model for the provider	Unset <code>LLM_MODEL</code> to use the provider default, or set a valid one.
Azure <code>404</code> <code>Resource not found</code>	Trailing comma/quote in endpoint, deployment, or key	Strip trailing punctuation from <code>AZURE_OPENAI_*</code> values in <code>.env</code> .
429 / rate limited	Provider quota (account-level)	Higher tier, multiple keys, fewer multiqueries/revisions; <code>LLM_MAX_RETRIES</code> covers transient only.
Memory lost across requests	<code>CHECKPOINTINTER=memory</code> with multiple replicas/restarts	Switch to <code>sqlite</code> (1 node) or <code>postgres</code> (many).
Readiness probe never passes	Embedding model still downloading / KB unreachable	Raise <code>initialDelaySeconds</code> ; pre-warm in image (already done in Dockerfile).
High latency, low CPU	LLM-bound	Cheaper supervisor/critic model; reduce <code>MULTIQUERY_COUNT</code> / <code>MAX_REVISIONS</code> ; add cache.
Critic loops to <code>MAX_REVISIONS</code> every time	Retrieval not surfacing supporting context	↑ <code>RETRIEVER_FETCH_K</code> , enable rerank, improve corpus; inspect via LangSmith trace.

13. Production readiness checklist

Config

- ☐ `AGENT_MODE=multiagent`
- ☐ `ENVIRONMENT=production`
- ☐ Provider key set, `LLM_MODEL` validated

State

- ☐ `CHECKPOINTER=postgres` (multi-replica)
- ☐ `POSTGRES_DSN` to managed DB
- ☐ Image built with `--extra postgres`

Security

- ☐ `API_KEYS` set & rotated
- ☐ `CORS_ORIGINS` allow-list
- ☐ TLS at ingress · non-root container

Resilience

- ☐ Rate limit on
- ☐ Timeouts aligned
- ☐ HPA min/max set · probes wired

Observability

- ☐ `/metrics` scraped
- ☐ JSON logs shipped
- ☐ LangSmith tracing in staging

Retrieval

- ☐ KB ingested in image/volume
- ☐ Shared vector store for replicas
- ☐ Reranker sizing decided

Verified path. This pipeline has been run live (Azure OpenAI): `supervisor` → `retrieval (chunks)` → `synthesis` → `critic (grounded=True)` producing a cited, grounded answer — with 74 offline tests, ruff, and mypy all green. The deployment artifacts above are the same ones the project ships (`Dockerfile` , `docker-compose.yml` , CI).

Related documents

- [Multi-Agent RAG — Architecture & Guide](#) — the agents, graph, and retrieval internals.
- [Production-Grade Guide](#) — broader deployment, scaling, and security reference.
- [Operations Runbook](#) — day-2 operations for the service.
- [Architecture](#) — the overall project design.